SQL Injection

Description

La SQL Injection è una vulnerabilità in cui un'applicazione accetta input da un utente senza verificare che l'input non contenga ulteriori comandi SQL.

```
<?php
    $username = $_GET['username']; // kchung
    $result = mysql_query("SELECT * FROM users WHERE username
='$username'");
?>
```

Se osserviamo la variabile **$username**, in condizioni normali ci aspetteremmo che il parametro **username** sia un vero nome utente (ad es. *kchung*).

Ma un utente malintenzionato potrebbe inviare dati di tipo diverso. Ad esempio, considera se l'input fosse semplicemente `'`?

L'applicazione si bloccherebbe perché la query SQL risultante sarebbe errata:

```
SELECT * FROM users WHERE username='''
```

Nota la virgoletta singola in più alla fine.

Sapendo che una virgoletta singola causerà un errore nell'applicazione, possiamo approfondire ulteriormente il concetto di SQL Injection.

Cosa succederebbe se il nostro input fosse `' OR 1=1`?

```
SELECT * FROM users WHERE username='' OR 1=1
```

Il numero **1** è effettivamente uguale a **1**. Questo equivale a vero in SQL.

Se riconsideriamo la situazione, la query SQL in realtà dice:

```
SELECT * FROM users WHERE username='' OR true
```

Questo restituirà ogni riga della tabella perché ogni riga esistente risulta vera.

Possiamo anche iniettare commenti e caratteri di terminazione come `--`, `/*` o `;`. Questo permette di terminare una query SQL dopo le istruzioni iniettate. Ad esempio, `'--` è un payload comune per SQL injection.

```
SELECT * FROM users WHERE username=''-- '
```

Questo payload imposta il parametro **username** a una stringa vuota per uscire dalla query e poi aggiunge un commento (`--`) che nasconde efficacemente la seconda virgoletta singola.

Utilizzando questa tecnica di aggiungere istruzioni SQL a una query esistente, possiamo costringere i database a restituire dati che non avrebbero dovuto essere restituiti.

Tipi di Attacchi SQL Injection

Esistono diversi tipi di attacchi SQL injection, che differiscono per complessità e gravità:

---

1. SQL Injection in-band (SQLi Classica)

La SQL injection in-band si verifica quando l'attaccante può sia iniettare il codice SQL che recuperare i risultati della query attraverso lo stesso canale di comunicazione (ad esempio, il sito web).

- **SQL Injection basata sugli errori**:

  In questo caso, gli attaccanti inducono il database a generare errori che espongono informazioni sulla struttura del database. Ad esempio, un attaccante potrebbe deliberatamente provocare un errore per rivelare i nomi delle colonne o la struttura delle tabelle.

  Esempio:

  ```
  ' OR 1=1--
  ```

  Se viene restituito un errore, questo potrebbe rivelare informazioni sullo schema del database.

- **SQL Injection basata su UNION**:

  L'attaccante utilizza l'operatore SQL `UNION` per combinare i risultati di due istruzioni SELECT. Iniettando ulteriori query SELECT, può estrarre dati da altre tabelle del database.

  Esempio:

  ```
  ' UNION SELECT username, password FROM users--
  ```

  Questa query restituirà sia i risultati della query originale che i valori della tabella `users`.

---

2. Blind SQL Injection

La Blind SQL injection viene utilizzata quando l'applicazione web non restituisce errori o risultati in modo diretto. Invece, l'attaccante invia un payload e osserva il comportamento dell'applicazione (ad esempio, i tempi di risposta, eventuali reindirizzamenti o il contenuto della pagina) per dedurre l'esito della query.

- **Blind SQL Injection basata su valori booleani**:

  L'attaccante invia query che restituiscono un risultato vero o falso. In base alla risposta (se la pagina cambia o meno), può dedurre la struttura del database.

  Esempio:

  ```
  ' OR 1=1--
  ```

  Se la query restituisce vero, la pagina verrà caricata normalmente; altrimenti, la pagina potrebbe comportarsi in modo diverso.

- **Blind SQL Injection basata sul tempo**:

  L'attaccante inietta codice SQL che induce il database a ritardare la risposta per un determinato periodo, confermando così se la query è stata eseguita correttamente o meno.

  Esempio:

```
' OR IF(1=1, SLEEP(5), 0)--
```

Se la pagina impiega 5 secondi a caricarsi, l'attaccante capisce che la query ha restituito vero.

---

3. SQL Injection Out-of-band

La SQL injection out-of-band è meno comune e si verifica quando l'attaccante utilizza un canale di comunicazione diverso (come richieste HTTP) per recuperare i dati dal server. Questo metodo può essere usato quando il server è troppo lento o non restituisce risposte dirette, ma può innescare azioni esterne (come l'invio di richieste HTTP o email).

Conseguenze degli Attacchi SQL Injection

La SQL injection può avere conseguenze gravi per le applicazioni web e i loro utenti:

1. **Furto di Dati**:Gli attaccanti possono recuperare dati sensibili, inclusi username, password, email e dettagli finanziari.
2. **Manipolazione dei Dati**:Gli attaccanti possono modificare, inserire o eliminare record nel database, portando alla corruzione dei dati.
3. **Bypass dell'Autenticazione**:La SQL injection può permettere agli attaccanti di accedere come qualsiasi utente, compresi gli amministratori, senza credenziali valide.
4. **Denial of Service (DoS)**:Gli attaccanti possono eseguire query che esauriscono le risorse del server, potenzialmente causando il crash dell'applicazione o del database.
5. **Esecuzione di Codice Remota**:Nei casi più gravi, gli attaccanti possono ottenere il controllo completo del server eseguendo comandi sul database.

Prevenire la SQL Injection

Per mitigare le vulnerabilità dovute alla SQL injection, gli sviluppatori dovrebbero adottare diverse best practices e misure difensive:

1. Utilizzare Prepared Statements (Query Parametrizzate)

I prepared statements garantiscono che l'input dell'utente venga trattato come dati e non come codice eseguibile. Questo è il metodo più efficace per prevenire la SQL injection. I prepared statements scappano automaticamente i caratteri pericolosi presenti nell'input, impedendo l'esecuzione di codice SQL iniettato.

Esempio in PHP (utilizzando PDO):

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?
AND password = ?');
$stmt->execute([$username, $password]);
```

Esempio in Python (utilizzando MySQL connector):

```
cursor.execute("SELECT * FROM users WHERE username = %s AND p
assword = %s", (username, password))
```

2. Validazione e Sanitizzazione dell'Input

Valida e sanitizza sempre l'input dell'utente per assicurarti che rispetti il formato atteso (ad esempio, un'email valida, senza caratteri speciali). Sebbene questo da solo non sia sufficiente, rappresenta un ulteriore livello di protezione.

3. Utilizzare Framework ORM (Object Relational Mapping)

I framework ORM mostrano le query SQL grezze, convertendole in un linguaggio di programmazione di livello superiore. Questo può ridurre il rischio di SQL injection impedendo agli sviluppatori di scrivere query SQL raw.

Esempio in Django (Python ORM):

```
user = User.objects.get(username=username, password=password)
```

4. Escapare l'Input dell'Utente

Se devi utilizzare query SQL dinamiche, assicurati di escapare correttamente i caratteri speciali (come le virgolette (`'`) o i punti e virgola (`;`)) presenti nell'input.

Tuttavia, questo metodo è soggetto a errori e meno sicuro rispetto all'utilizzo dei prepared statements.

5. Utilizzare Account Database con Privilegi Minimi

Utilizza account del database con i privilegi minimi necessari per il funzionamento dell'applicazione. In questo modo, anche se un attaccante riesce ad eseguire delle query, i danni saranno limitati.

6. Firewall per Applicazioni Web (WAF)

Un WAF può aiutare a filtrare e bloccare i tentativi di SQL injection ispezionando il traffico in entrata e impedendo alle richieste malevole di raggiungere l'applicazione.

Strumenti per il Test della SQL Injection

- **SQLMap**:

  Uno strumento open-source molto popolare che automatizza la rilevazione e lo sfruttamento delle vulnerabilità di SQL injection. Fornisce anche funzionalità per estrarre dati dal database, modificarli e persino eseguire comandi sul sistema operativo sottostante.

  Esempio:

  ```
  sqlmap -u "http://example.com/vulnerable_page.php?id=1"
  ```

- **Havij**:

  Uno strumento automatizzato per la SQL injection dotato di interfaccia grafica (GUI). Semplifica l'uso della SQL injection per utenti meno esperti.

- **jSQL**:

  Uno strumento di SQL injection basato su Java che aiuta i tester a identificare e sfruttare le vulnerabilità di SQL injection.